

Chapter 2: TypeScript for Angular

1. Overview

Angular is written in TypeScript, and its entire framework contract — component metadata, dependency injection, template type-checking, module boundaries — is expressed through TypeScript language features. You cannot reason about Angular correctly without a precise mental model of TypeScript's type system and how it compiles down to JavaScript. This is also one of the highest-yield areas in Angular interviews: interviewers routinely test decorators, generics, strict mode, access modifiers, and utility types not as abstract TypeScript trivia, but as the mechanism behind `@Component`, `@Input`, `Injectable`, RxJS typings, and reactive forms.

This chapter covers TypeScript strictly through the lens of "what an Angular developer must know cold": the type system (structural typing, unions/intersections, generics), decorators and how `experimentalDecorators / emitDecoratorMetadata` power Angular's DI, access modifiers and their *compile-time-only* nature, enums and `const enum` pitfalls, utility types used throughout Angular's own typings (`Partial`, `Pick`, `Record`, `ReturnType`), the module system (ES modules vs CommonJS, `isolatedModules`, tree-shaking implications), and the full compilation pipeline from `.ts` → `.js` via `tsc / ngc` (Ivy AOT).

2. Core Concepts

2.1 TypeScript's Type System Foundations

Structural typing (duck typing), not nominal typing. Two types are compatible if their shapes match, regardless of declared name or inheritance:

```
interface Point { x: number; y: number; }
class Vector { constructor(public x: number, public y: number) {} }

function log(p: Point) { console.log(p.x, p.y); }
log(new Vector(1, 2)); // OK - Vector's shape satisfies Point
```

This is why Angular's `HttpClient.get<T>()` and RxJS operators accept plain objects interchangeably with class instances, and why interfaces (not classes) are the idiomatic contract type for DTOs/models in Angular apps.

interface vs type alias.

- `interface` supports declaration merging (multiple declarations with the same name merge their members) and is the conventional choice for object shapes that might be extended (component `@Input` bag shapes, service contracts).
- `type` alias is required for unions, tuples, mapped/conditional types, and primitives aliasing.
- Both support extension: `interface B extends A` vs `type B = A & { ... }`.
- Interview nuance: `interface` can be `implements`-ed by a class; `type` (for object shapes) can also be implemented, but union/conditional `type`s cannot.

```
interface User { id: number; name: string; }
interface User { email: string; } // merges – User now has id, name, email

type Status = 'idle' | 'loading' | 'success' | 'error'; // only possible as a type alias
```

Union types (|) represent "one of several types" — the backbone of Angular's discriminated-union state modeling and RxJS type inference (e.g., `Observable<string | null>`).

Intersection types (&) combine multiple types into one that must satisfy all of them — used heavily for mixin patterns and combining `@Input` / `@Output` shapes.

```
type Timestamped = { createdAt: Date };
type Named = { name: string };
type Entity = Timestamped & Named; // must have both createdAt and name
```

Discriminated unions — the standard pattern for typing NgRx actions, HTTP resource states, and API responses safely:

```
type LoadState<T> =
  | { status: 'idle' }
  | { status: 'loading' }
  | { status: 'success'; data: T }
  | { status: 'error'; error: string };

function render<T>(state: LoadState<T>) {
  switch (state.status) {
    case 'success': return state.data; // TS narrows to the 'success' branch
    case 'error':   return state.error; // narrowed to 'error' branch
    default:        return null;
  }
}
```

Type narrowing — `typeof`, `instanceof`, `in`, truthiness checks, and user-defined type guards (`x is Foo`) let TypeScript refine a union to a specific member within a branch. Angular's `*ngIf` and template control flow (`@if`) do NOT automatically narrow types in templates the way they do in `.ts` code (a common gotcha — see §5).

2.2 Generics

Generics parameterize types, functions, classes, and interfaces over an unspecified type resolved at the call site — Angular and RxJS are built on generics end to end (`Observable<T>`, `EventEmitter<T>`, `HttpClient.get<T>()`, `FormControl<T>`, `Injectable` factories).

```
class Repository<T extends { id: number }> {
  private items: T[] = [];
  add(item: T): void { this.items.push(item); }
  findById(id: number): T | undefined {
    return this.items.find(i => i.id === id);
  }
}

const repo = new Repository<User>();
```

Generic constraints (`extends`) restrict what a type parameter may be, enabling safe access to members:

```
function pluck<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}
```

Default generic parameters: `class EventEmitter<T = any> extends Subject<T>` — this is literally how Angular's own `EventEmitter` is declared, which is why an untyped `@Output() foo = new EventEmitter();` silently becomes `EventEmitter<any>`.

Generic component/service patterns in Angular:

```
@Injectable({ providedIn: 'root' })
export class CrudService<T extends { id: number }> {
  constructor(private http: HttpClient) {}
  getAll(url: string): Observable<T[]> { return this.http.get<T[]>(url); }
}
```

2.3 Decorators & Metadata

Decorators are functions applied to classes, methods, accessors, properties, or parameters using the `@expression` syntax, evaluated at **class definition time** (not instantiation time). Angular relies on five decorator categories:

- **Class decorators:** `@Component`, `@Directive`, `@Injectable`, `@NgModule`, `@Pipe`.
- **Property decorators:** `@Input`, `@Output`, `@ViewChild`, `@ContentChild`.
- **Method decorators:** `@HostListener`.
- **Parameter decorators:** `@Inject`, `@Optional`, `@Self`, `@SkipSelf`, `@Host` — used inside constructors for DI hints.
- **Accessor decorators:** rare in Angular but valid on getters/setters.

A decorator is just a function that receives the target and returns (optionally) a replacement:

```
function Component(config: ComponentConfig) {
  return function (target: Function) {
    Reflect.defineMetadata('annotations', [config], target);
  };
}
```

This requires two `tsconfig.json` compiler flags:

```
{
  "experimentalDecorators": true,
  "emitDecoratorMetadata": true
}
```

`emitDecoratorMetadata` makes `tsc` emit design-time type metadata (parameter types, return type, property type) via calls to `Reflect.metadata`, using the `reflect-metadata` polyfill. **This is exactly how Angular's dependency injector used to determine constructor parameter types** for classic (pre-Ivy) DI resolution — `tsc` recorded `design:paramtypes` and Angular's compiler read it off the class at runtime/compile time to know "this constructor wants an `HttpClient`". With Ivy, Angular's own compiler (`ngtsc`) generates `efac` factory functions from the AST directly at compile time rather than relying purely on reflected metadata, but decorators are still the *source of truth* the compiler statically analyzes.

Decorators execute **bottom-to-top, and for multiple decorators on one declaration, evaluate top-to-bottom but invoke bottom-to-top** (like function composition):

```
@first()
@second()
class Example {}
// evaluation order: first(), second()
// invocation (application) order: second's result wraps first's – second applied, then first
```

Decorator factories — `@Input()` is not a decorator itself, it's a function call that *returns* a decorator, allowing configuration (`@Input('aliasName')`, `@Input({ required: true })` in newer Angular).

2.4 Access Modifiers & Parameter Properties

`public`, `private`, `protected`, and `readonly` are **compile-time-only** constructs — TypeScript strips them entirely during transpilation (except for the newer native `#private` fields, and `readonly`, which has zero runtime enforcement either way). At runtime, a "private" class member in compiled JS is fully accessible unless you use ECMAScript's native `#field` private syntax.

```
class UserService {
  constructor(
    private http: HttpClient,    // parameter property: declares + assigns `this.http`
    protected logger: Logger,
    readonly apiUrl: string,
    public config: Config
  ) {}
}
```

This constructor shorthand ("parameter properties") is idiomatic in every Angular service/component constructor — it both declares the class field and assigns the injected instance in one line.

- `private`: accessible only within the declaring class.
- `protected`: accessible within the declaring class and subclasses — commonly used for members a base component class exposes to its children (e.g., abstract base form components).

- `public` (default): accessible anywhere.
- `readonly`: assignable only at declaration or inside the constructor; enforced only by the type checker.
- Native `#field`: real runtime privacy (introduced via ES2022 private fields), enforced by the JS engine, not erasable, and not visible via `Object.keys`/reflection.

```
class Counter {
  #count = 0;           // truly private at runtime
  private legacy = 0;    // only private at compile time – visible in emitted JS
  increment() { this.#count++; }
}
```

Angular-specific nuance: template bindings can only access members that are `public` from the *component class*'s perspective as seen by the compiler; Angular's Ivy template type-checker (in `--strictTemplates` mode) will flag template expressions accessing `private`/`protected` members with a compile error, even though at runtime nothing would stop it, because the generated template-check code lives in a different lexical/module scope that the type checker treats as external.

2.5 Enums

```
enum Direction { Up, Down, Left, Right } // numeric enum: Up=0, Down=1, ...
enum Status { Active = 'ACTIVE', Inactive = 'INACTIVE' } // string enum
```

- Numeric enums are **bidirectional** at runtime — `Direction[0] === 'Up'` and `Direction.Up === 0` both work, because TS emits a reverse-mapping object.
- String enums have **no reverse mapping** and produce more readable emitted JS/debugging output — the general recommendation for Angular apps (e.g., typing route data, form states) is to prefer string enums or union-of-string-literals over numeric enums.
- `const enum` is fully inlined at compile time (no object is emitted at all) — but it's **incompatible with `isolatedModules: true`**, which Angular CLI's default build (via esbuild/SWC-based single-file transpilation) generally requires, so `const enum` is effectively banned in standard Angular projects.
- Many teams prefer a union-of-literals + `as const` object over `enum` entirely, since enums are a TS-only nominal construct that don't tree-shake as cleanly and don't interoperate as smoothly with plain JS/JSON APIs:

```
const Status = { Active: 'ACTIVE', Inactive: 'INACTIVE' } as const;
type Status = typeof Status[keyof typeof Status]; // 'ACTIVE' | 'INACTIVE'
```

2.6 Strict Mode

`"strict": true` in `tsconfig.json` (Angular CLI's default since v9+ for new projects) bundles several flags, each independently toggleable:

Flag	Effect
<code>strictNullChecks</code>	<code>null</code> / <code>undefined</code> are not assignable to other types unless explicitly unioned in. The single most impactful flag for Angular apps — forces explicit handling of <code>@Input() value?: string</code> vs <code>value!: string</code> .
<code>noImplicitAny</code>	Disallows variables/parameters inferred as <code>any</code> without an explicit annotation.
<code>strictFunctionTypes</code>	Enforces contravariant checking of function parameter types.
<code>strictPropertyInitialization</code>	Class properties must be initialized in the constructor or have a definite assignment assertion (<code>!</code>) or be optional (<code>?</code>) — this is why Angular <code>@Input() name!: string</code> ; uses the <code>!</code> (definite assignment assertion) so heavily, since inputs are set by Angular <i>after</i> construction, not in the constructor.
<code>strictBindCallApply</code>	Type-checks arguments to <code>.bind()</code> , <code>.call()</code> , <code>.apply()</code> .
<code>noImplicitThis</code>	Flags <code>this</code> with an implicit <code>any</code> type.
<code>alwaysStrict</code>	Emits <code>"use strict"</code> and parses in strict ECMAScript mode.

Angular additionally exposes its own `strictTemplates` flag (in `angularCompilerOptions`) which is not a raw TS flag but instructs Ivy's template type-checker to apply full strict type-checking (including `strictNullChecks`) to template expressions, `@Input()` bindings, and structural directive contexts.

```
{
  "compilerOptions": { "strict": true },
  "angularCompilerOptions": { "strictTemplates": true, "fullTemplateTypeCheck": true }
}
```

2.7 Utility Types

TypeScript's built-in utility types are used extensively both inside Angular's own `.d.ts` files and in idiomatic app code:

```
interface User { id: number; name: string; email: string; age: number; }

type PartialUser = Partial<User>;           // all properties optional – great for PATCH
payloads / form patchValue()
type ReadonlyUser = Readonly<User>;         // immutable view, e.g., NgRx state slices
type UserPreview = Pick<User, 'id' | 'name'>; // subset of keys
type UserWithoutEmail = Omit<User, 'email'>; // all but specified keys
type UserRecord = Record<number, User>;     // dictionary keyed by id
type RequiredUser = Required<User>;         // opposite of Partial – all keys mandatory
type NameOrId = Pick<User, 'name'> | Pick<User, 'id'>;
type UserKeys = keyof User;                 // 'id' | 'name' | 'email' | 'age'

// Function-related utilities, common when wrapping RxJS/service methods:
```

```
function getUser(): Observable<User> { /* ... */ return of({} as User); }
type GetUserReturn = ReturnType<typeof getUser>; // Observable<User>

class ApiService {
  fetch(url: string, opts: { retries: number }): void {}
}
type FetchParams = Parameters<ApiService['fetch']>; // [string, { retries: number }]

type MaybeUser = User | null;
type DefinitelyUser = NonNullable<MaybeUser>; // User
```

Also common: **mapped types** (the mechanism behind `Partial` / `Readonly` etc. under the hood) and **conditional types**:

```
type Nullable<T> = { [K in keyof T]: T[K] | null };

type ElementType<T> = T extends (infer U)[] ? U : never;
type Item = ElementType<string[]>; // string
```

2.8 Module System

Angular apps are authored using **ES module** syntax (`import` / `export`), which TypeScript type-checks and then transpiles per the `module` compiler option:

- `"module": "es2020" / "esnext"` — preserves ES module syntax in output, required for **tree-shaking** by bundlers (webpack/esbuild) since static `import / export` is statically analyzable, unlike CommonJS `require`.
- `"moduleResolution": "node" / "bundler"` — governs how import specifiers are resolved to files.
- `isolatedModules: true` — required by Angular CLI's modern build pipeline (esbuild-based `@angular/build`, and Babel/SWC-style single-file transpilers) because each file must be transpilable independently without cross-file type information — this is why `const enum`, and re-exporting types without `export type`, are disallowed under this flag.
- `export type { Foo } / import type { Foo }` — explicitly marks a type-only import/export so it is guaranteed to be fully erased and never emitted as a runtime `import`, which matters under `isolatedModules`.

Angular's own package format (Angular Package Format / APF) ships both an ES module build (`.mjs` / ESM for tree-shaking) and type declaration files (`.d.ts`), which is how `ng build --prod` can tree-shake unused Angular framework code (e.g., unused RxJS operators, unused Angular Material components) out of the final bundle.

3. Code Examples

3.1 A fully realistic component leaning on TS features

```
import { Component, Input, Output, EventEmitter, OnInit, OnChanges, SimpleChanges } from
  '@angular/core';

type Role = 'admin' | 'editor' | 'viewer';
```

```

interface UserCardModel {
    id: number;
    name: string;
    role: Role;
    metadata?: Record<string, unknown>;
}

@Component({
    selector: 'app-user-card',
    standalone: true,
    template: `<div>{{ user.name }} - {{ user.role }}</div>`
})
export class UserCardComponent implements OnInit, OnChanges {
    @Input({ required: true }) user!: UserCardModel;    // definite assignment assertion
    @Input() readonly compact: boolean = false;
    @Output() roleChanged = new EventEmitter<Role>();

    private readonly allowedRoles: readonly Role[] = ['admin', 'editor', 'viewer'] as const;

    ngOnInit(): void {
        this.validateRole(this.user.role);
    }

    ngOnChanges(changes: SimpleChanges): void {
        if (changes['user'] && !changes['user'].firstChange) {
            this.validateRole(this.user.role);
        }
    }

    private validateRole(role: Role): asserts role is Role {
        if (!this.allowedRoles.includes(role)) {
            throw new Error(`Invalid role: ${role}`);
        }
    }

    promote(next: Role): void {
        this.roleChanged.emit(next);
    }
}

```

3.2 Generic service with constrained type parameter

```

interface Identifiable { id: number; }

@Injectable({ providedIn: 'root' })
export class EntityStore<T extends Identifiable> {
    private readonly entities = new Map<number, T>();

    upsert(entity: T): void {
        this.entities.set(entity.id, entity);
    }
}

```



```
select<K extends keyof T>(id: number, key: K): T[K] | undefined {
  return this.entities.get(id)?.[key];
}
}
```

3.3 Discriminated union modeling an HTTP resource (typical NgRx / signal-store pattern)

```
type Resource<T> =
  | { kind: 'idle' }
  | { kind: 'loading' }
  | { kind: 'loaded'; value: T }
  | { kind: 'failed'; error: unknown };

function isLoading<T>(r: Resource<T>): r is { kind: 'loaded'; value: T } {
  return r.kind === 'loaded';
}
```

3.4 Custom decorator (illustrating the mechanism Angular builds on)

```
function LogInvocation(): MethodDecorator {
  return function (target, propertyKey, descriptor: PropertyDescriptor) {
    const original = descriptor.value;
    descriptor.value = function (...args: unknown[]) {
      console.log(`Calling ${String(propertyKey)} with`, args);
      return original.apply(this, args);
    };
    return descriptor;
  };
}

class OrderService {
  @LogInvocation()
  placeOrder(id: number): void { /* ... */ }
}
```

4. Internal Working

4.1 The `.ts` → `.js` pipeline

1. **Parsing:** `tsc` parses `.ts` / `.html` (Angular templates are parsed separately by Angular's own template parser) into an AST.
2. **Binding & type-checking:** TypeScript's checker walks the AST, resolves symbols, infers/validates types, and reports diagnostics — this step produces **no runtime output**; it's purely a static analysis pass.
3. **Transformation:** TypeScript-specific constructs are stripped or downleveled — interfaces, type annotations, `as` casts, and generics vanish entirely; enums, decorators (with `experimentalDecorators`), and class parameter properties are transformed into equivalent plain JS

(e.g., an enum becomes an IIFE-populated object; a decorator becomes a call to `__decorate` in the emitted helper).

4. **Emit:** plain JavaScript (`.js`) targeting the configured `target` (e.g., `ES2022`) and `module` format is written out, with a `.d.ts` declaration file if `declaration: true`.

Crucially: **types are fully erased** — there is no `typeof x === 'User'` at runtime; type information exists only to catch mistakes before compilation.

4.2 Angular's build pipeline vs raw `tsc`

Angular does not use raw `tsc` for application compilation — it uses `ngtsc` (the Angular/Ivy compiler), a `tsc`-compatible compiler that wraps the TypeScript compiler with an additional AST transformation pass:

1. `ngtsc` runs the normal TypeScript parse/check/transform pipeline.
2. During transformation, it detects Angular decorators (`@Component` , `@Injectable` , etc.) on classes.
3. For each decorated class it statically analyzes the decorator's metadata object (the object literal passed to `@Component({ ... })`) must be statically analyzable — this is why you cannot pass a dynamically computed template string built from an arbitrary function call) and **generates additional class members** directly into the compiled output:
 - `ɵcmp` (component definition — `ComponentDef` , produced by `defineComponent()`), containing the compiled template's render function, style URLs, change detection strategy, inputs/outputs metadata, etc.
 - `ɵfac` (factory function), describing how to construct the class and resolve constructor dependencies (this is the direct successor to what `emitDecoratorMetadata` + `reflect-metadata` used to provide reflectively — Ivy makes it static and tree-shakeable instead of reflection-based).
 - `ɵdir` , `ɵmod` , `ɵpipe` , `ɵprov` analogues for directives, modules, pipes, injectables.
4. Templates are parsed, and in `strictTemplates` mode, **type-checked against the component class** by generating a synthetic "type-check block" (TCB) function whose body mimics the template's expressions using real TypeScript syntax, so the ordinary TS checker can flag template type errors (e.g., binding a `string` to an `@Input()` `count: number`) using the same diagnostic machinery as regular code.
5. AOT (the default and only mode since Ivy) means this entire process — template compilation to render functions — happens at **build time**, not in the browser; the deployed bundle contains no template strings or Angular compiler at runtime for application code (a JIT compiler is no longer shipped by default).

4.3 Decorators and metadata reflection in detail

With `experimentalDecorators: true` , a decorator call like:

```
@Component({ selector: 'app-root', template: '...' })
export class AppComponent {}
```

compiles (conceptually) to:

```
let AppComponent = class AppComponent {};
AppComponent = __decorate([
  Component({ selector: 'app-root', template: '...' })
], AppComponent);
```

`__decorate` is a TypeScript-emitted helper that applies each decorator function to the class, in reverse (bottom-up) order, and — if `emitDecoratorMetadata` is on — also calls `Reflect.metadata('design:paramtypes', [...])` etc., attaching design-time type info that `reflect-metadata` stores for runtime retrieval. Pre-Ivy Angular (`ViewEngine`) actually depended on this reflected metadata (via `Reflect.getMetadata`) to resolve constructor DI; Ivy replaced this at compile time with statically generated `ɵfac` functions, meaning modern Angular **no longer strictly requires** `reflect-metadata` / `emitDecoratorMetadata` **for its own DI to function** — though some libraries (or `@Inject` edge cases) can still interact with it.

5. Edge Cases & Gotchas

- `strictNullChecks` **off masks real bugs**: without it, `string | null | undefined` collapses to interacting with plain `string` everywhere, and Angular templates silently render `undefined` as empty string instead of erroring at compile time — turning this off defeats one of TypeScript's biggest safety wins for Angular apps.
- **! (definite assignment assertion) is a promise, not a guarantee**: `@Input() name!: string` tells the compiler "trust me, this will be set" — if Angular never binds it (e.g., consumer forgets `[name]`), you get a runtime `undefined` with no compile-time warning.
- **Template expressions don't narrow the same way .ts code does** in older Angular versions/non-strict setups: `*ngIf="user"` narrows `user` inside the block reliably under `strictTemplates`, but complex custom type guards used inside templates are not always recognized — prefer computing narrowed values in the component class and exposing simple bindings.
- `private` / `protected` **are erased at runtime** — "encapsulation" from TS is a development-time contract only; anyone can access a `private` field from plain JS or via `(obj as any).secret`. Only `#field` is truly private.
- `const enum` **breaks under** `isolatedModules` (the default for Angular's esbuild-based build system) — a common migration surprise when moving old projects to modern Angular CLI builders.
- **Enums don't tree-shake as well as literal unions** — an imported `enum` pulls in its runtime object even if only one member is used; a `type` alias of string literals is fully erased and costs nothing at runtime.
- **Numeric enum reverse mapping is a footgun** for anything serialized to JSON/APIs — `JSON.stringify` for an object keyed by a numeric enum's reverse-mapped keys can produce unexpected extra entries if you iterate `Object.keys(EnumObj)`.
- **Structural typing can accept "too much"**: a function expecting `{ id: number }` will happily accept a `user` object with 20 extra fields (unless the object is an object literal assigned directly, in which case *excess property checking* kicks in and flags unknown properties) — this trips people up because a stored variable bypasses the check that a literal wouldn't.
- **this typing inside RxJS operators/callbacks and Angular lifecycle hooks**: arrow functions inherit lexical `this`, while regular `function` callbacks passed to `.subscribe()` or `setTimeout` lose the

component's `this` unless bound — a classic bug even with TypeScript's `noImplicitThis`, since `this` can still be explicitly (and wrongly) typed.

- **Decorator evaluation order surprises:** for a class with both a class decorator and property decorators, property/method decorators run before the class decorator; multiple decorators on the same declaration apply bottom-up (closest to the declaration first).
- **any vs unknown:** `any` disables type checking entirely (a hole in the type system); `unknown` is type-safe — it must be narrowed before use. Interviewers commonly probe whether candidates default to `any` out of habit; idiomatic modern Angular/TS code prefers `unknown` for values of truly unknown shape (e.g., generic error handlers, `catchError` payloads).
- **Object.freeze vs readonly:** `readonly` is compile-time only, so `JSON.parse(JSON.stringify(readonlyobj))` or direct manipulation through an `any`-cast bypasses it entirely; only `Object.freeze()` gives actual runtime immutability (and only shallowly).

6. Interview Questions & Answers

Q1. What is the difference between `interface` and `type` in TypeScript, and when would you choose one over the other in an Angular codebase?

A: Both can describe object shapes and support extension. `interface` supports declaration merging (re-opening the same interface name to add members) and is generally preferred for public object contracts (component `@Input` models, service DTOs) because it reads clearly in stack traces/IDE tooltips and can be implemented by classes. `type` is required for unions, intersections, tuples, mapped/conditional types, and primitive aliases — anything that isn't a plain object shape. A practical rule: use `interface` for the shape of "things" (models, service contracts), `type` for everything algebraic (unions of state, utility-type compositions).

Q2. Explain structural typing and why it matters for how Angular's `HttpClient` and `RxJS observables` behave.

Interviewer intent: checks whether the candidate understands TypeScript's core type-compatibility model rather than assuming Java/C#-style nominal typing.

A: TypeScript uses structural (shape-based) type compatibility: a value is assignable to a type if it has at least the required members with compatible types, regardless of its declared class or name. This means `http.get<User>('/api/user')` type-checks the *response shape*, not its runtime identity — a plain JSON object satisfies a `User` interface with no class instantiation needed. It's also why mock objects and interfaces (rather than requiring concrete classes) work seamlessly for testing services and components.

Q3. What does `strictNullChecks` actually change, and why is it important in Angular applications specifically?

A: Without it, `null` and `undefined` are assignable to every type, so `let name: string = null` type-checks and any access chain is implicitly unsafe. With it on, a type must explicitly include `| null` / `| undefined` to accept those values, forcing you to guard before use (`user?.name`, `if (user) {...}`). In Angular apps this matters enormously because `@Input()` values, `@ViewChild()` query results, and route params are frequently `undefined` until a certain lifecycle point (e.g., `@ViewChild` is `undefined` until `ngAfterViewInit`), and `strictNullChecks` forces you to handle that instead of getting a runtime `Cannot read property of undefined`.

Q4. Why does `@Input() name!: string;` use the `!` operator, and what would

`strictPropertyInitialization` complain about without it?

A: `strictPropertyInitialization` requires every class property to be assigned a value by the time the constructor finishes (or be optional / have a default). But `@Input()` values are assigned by Angular's change detection *after* construction, not in the constructor — so if you write `@Input() name: string;` with strict mode on, TypeScript errors because it sees an uninitialized required property. The `!` is a "definite assignment assertion" telling the compiler "I guarantee this will be assigned before use, trust me" — it silences the error without adding runtime safety; if the input is genuinely never bound, you'll get `undefined` at runtime with no complaint from TS.

Q5. How do decorators work under the hood, and what two compiler flags are required for Angular's decorators to function?

Interviewer intent: tests whether the candidate can go one level beneath "decorators are annotations" into the actual compiled output and metadata mechanism.

A: A decorator is just a function invoked at class-definition time that receives the target (class, prototype, or descriptor) and can inspect or replace it. Angular requires `experimentalDecorators: true` (enables the `@` syntax transform, compiling decorator usage into calls to a `__decorate` helper) and `emitDecoratorMetadata: true` (makes `tsc` also emit `design:type` / `design:paramtypes` / `design:returntype` metadata via `Reflect.metadata` calls, readable at runtime through the `reflect-metadata` polyfill). Historically (ViewEngine), Angular's DI used this reflected metadata to figure out constructor dependency types; with Ivy, the compiler (`ngtsc`) statically generates factory functions (`@fac`) from the AST at compile time instead, so decorators remain the mechanism the compiler recognizes and analyzes, but runtime reflection is no longer strictly required for DI to work.

Q6. Are `private` and `protected` enforced at runtime in TypeScript? What actually happens to them when compiled to JavaScript?

A: No — they are compile-time-only constructs used purely by the type checker. When TypeScript transpiles to JavaScript, `private` / `protected` / `public` keywords are stripped entirely; the resulting class member is a completely ordinary, publicly accessible property or method. Only the newer native `#fieldName` private class fields (an ECMAScript feature, not TypeScript-specific) are enforced at the JS engine level and cannot be accessed or even observed (e.g., via `Object.keys` or bracket notation) from outside the class.

Q7. What is a generic constraint, and how would you write a generic `EntityStore<T>` class that only accepts types with an `id` field?

A: A generic constraint restricts what types can be substituted for a type parameter, using `extends`. You'd write `class EntityStore<T extends { id: number }> { ... }`, which lets the class body safely assume every `T` has a numeric `id` (e.g., to use as a Map key) while still being generic over the rest of the shape. Attempting `new EntityStore<{ name: string }>()` (missing `id`) would fail to compile.

Q8. What's the difference between `Partial<T>`, `Pick<T, K>`, and `Omit<T, K>`, and give a realistic Angular use case for each.

A: `Partial<T>` makes every property of `T` optional — useful for PATCH-style update payloads or `FormGroup.patchValue()` where you only supply a subset of a model's fields. `Pick<T, K>` constructs a type with only the listed keys from `T` — useful for a lightweight "preview" or "list row" view model derived from a larger entity interface without duplicating field definitions. `Omit<T, K>` is the inverse — all of `T`'s keys except the listed ones — commonly used to derive a "create" DTO type that excludes a server-generated `id` field from a full entity interface.

Q9. Explain the difference between `any` and `unknown`, and why `unknown` is considered the safer default.

A: `any` opts a value entirely out of the type system — any operation on it (property access, calling it, arithmetic) is allowed without any checking, effectively creating a hole that can propagate runtime errors silently past the compiler. `unknown` also accepts any value, but the compiler forbids using it in any way until you narrow it (via `typeof`, `instanceof`, a type guard, or an explicit assertion). It's the correct type for things like generic HTTP error handlers or `catch` blocks (`catch (e: unknown)`), where you genuinely don't know the shape in advance but still want the compiler to force you to check before use.

Q10. Numeric enums vs string enums vs union-of-string-literal types — what's the tradeoff, and which would you pick for typing something like a user role in an Angular app?

Interviewer intent: probes whether the candidate has actually hit the tree-shaking/serialization issues with enums in production, not just textbook syntax knowledge.

A: Numeric enums get a bidirectional reverse mapping emitted at runtime (`Enum[0] === 'Name'` and vice versa), which bloats output and can leak unexpected keys if you iterate `object.keys()` on the enum object. String enums avoid the reverse mapping and are more debuggable (values print as meaningful strings), but both are nominal TS-only constructs that don't tree-shake as cleanly as literal types and require an explicit import wherever used. A union of string literals (`type Role = 'admin' | 'editor' | 'viewer'`) is fully erased at compile time — zero runtime cost, no reverse mapping, trivially JSON-serializable, and works identically well with a companion `as const` object if you need enumerable values at runtime. For something like a user role, I'd default to the literal union (optionally paired with a `const` object of values) unless the team specifically wants the enum's namespacing or is integrating with a library that expects real enum types.

Q11. Why can't Angular's `@Component` decorator accept a dynamically computed configuration object, e.g., `@Component(getConfig())`?

A: Angular's compiler (`ngtsc`) needs to statically analyze the decorator's argument at compile time to extract the template, styles, selector, and other metadata and generate the `ɵcmp` definition — this happens during compilation, before any code runs. If the argument were the result of an arbitrary function call, the compiler couldn't evaluate it without actually executing your TypeScript program (which it doesn't do at compile time), so it can't extract stable metadata deterministically. Angular's AOT compiler therefore requires the decorator's configuration object to be a statically analyzable object literal (constants, template strings/`templateur1`, literal arrays are fine; arbitrary runtime computation is not).

Q12. What does `isolatedModules: true` require of your code, and why does Angular's modern build system need it?

A: `isolatedModules` requires that every file can be transpiled to JavaScript independently, without needing type information from other files — this is necessary because fast, parallelizable, single-file transpilers (esbuild, SWC, Babel) don't do full cross-file type checking, they just strip types file by file. Under this flag, you cannot use `const enum` (its inlining requires knowing enum member values from another file), and re-exporting a type must use `export type { Foo }` rather than plain `export { Foo }` (otherwise the transpiler can't tell if `Foo` is a type, which should be erased, or a value, which must be preserved, without consulting the original file). Angular's CLI build pipeline (the `@angular/builder/esbuild-based` builder) relies on this kind of fast, isolated per-file transpilation, so `isolatedModules` is effectively mandatory in modern Angular projects.

Q13. What is a discriminated union, and how would you model an HTTP resource's loading/success/error states using one?

Interviewer intent: checks whether the candidate reaches for type-safe state modeling instead of loose booleans (`isLoading`, `hasError`, `data`) that can enter impossible combinations.

A: A discriminated union is a union of object types that share a common literal "tag" property (often called

`kind`, `type`, or `status`), which TypeScript uses to narrow the union to a specific member inside conditional branches. Modeling `type Resource<T> = { status: 'idle' } | { status: 'loading' } | { status: 'success'; data: T } | { status: 'error'; error: string }` makes illegal states unrepresentable — you cannot have `data` without `status: 'success'`, unlike a flat `{ isLoading: boolean; hasError: boolean; data?: T }` shape where `isLoading && hasError` simultaneously true is possible but meaningless. Inside a `switch (resource.status)`, TypeScript narrows `resource` to the matching branch's exact shape, giving compile-time-safe access to `resource.data` or `resource.error` only where valid.

Q14. How does Angular's Ivy compiler use TypeScript's compiler API, and what gets generated on a decorated class as a result?

A: `ngtsc` (Ivy's compiler) is built as a wrapper/plugin around the standard TypeScript compiler, reusing its parsing, binding, and type-checking phases, then adding an additional AST transformation step. During that step, it recognizes Angular decorators on classes and statically analyzes their (necessarily literal) configuration objects, then injects additional static class members into the compiled output: `ɵcmp` / `ɵdir` / `ɵmod` / `ɵpipe` / `ɵprov` (definition objects consumed by Angular's runtime, containing the compiled template render function, inputs/outputs, selectors, providers, etc.) and `ɵfac` (a factory function encoding how to construct the class and resolve its dependencies, replacing the old `emitDecoratorMetadata` + reflection-based approach with something fully static and tree-shakeable). In `strictTemplates` mode, it additionally generates a synthetic "type-check block" per template so the ordinary TypeScript checker can validate template bindings against the component class's real types.

7. Quick Revision Cheat Sheet

- **Structural typing:** shape matters, not declared name/class — enables interfaces to be satisfied by plain objects.
- **interface vs type:** interface merges declarations & is implementable; type is required for unions/intersections/conditional/mapped types.
- **Union (|)** = one of; **Intersection (&)** = all of; **discriminated unions** use a common literal tag for safe narrowing.
- **Generics:** `<T>`, constraints via `extends`, defaults via `= any`; power `Observable<T>`, `EventEmitter<T>`, `HttpClient.get<T>()`, `FormControl<T>`.
- **Decorators:** functions run at class-definition time; need `experimentalDecorators` (+ `emitDecoratorMetadata` for reflected type info via `reflect-metadata`); Angular's Ivy compiler statically reads decorator config to emit `ɵcmp` / `ɵfac` / etc. rather than relying purely on runtime reflection.
- **Access modifiers:** `private` / `protected` / `public` / `readonly` are compile-time only, erased on emit; only native `#field` is runtime-private.
- **Parameter properties:** `constructor(private http: HttpClient)` declares + assigns in one line — the standard Angular DI pattern.
- **Enums:** numeric enums get bidirectional reverse mapping; string enums don't; `const enum` is fully inlined but incompatible with `isolatedModules`; literal unions are often the leaner, tree-shakeable alternative.
- **Strict mode:** `strict: true` bundles `strictNullChecks`, `noImplicitAny`, `strictPropertyInitialization`, etc. — `strictNullChecks` is the highest-impact flag for Angular apps; `strictPropertyInitialization` is why `@Input() x!: T` needs the `!`.
- **strictTemplates:** Angular-specific flag (in `angularCompilerOptions`) enabling full strict type-checking

of template expressions/bindings via Ivy's generated type-check blocks.

- **Utility types:** `Partial`, `Required`, `Readonly`, `Pick`, `Omit`, `Record`, `ReturnType`, `Parameters`, `NonNullable`, `keyof` — all mapped/conditional types under the hood.
- **any vs unknown:** `any` disables checking entirely; `unknown` forces narrowing before use — prefer `unknown` for genuinely unknown external data.
- **Modules:** ES module syntax (`import / export`) is required for tree-shaking; `isolatedModules: true` (needed by Angular's esbuild-based CLI builder) forbids `const enum` and bare type re-exports (use `export type`).
- **Compilation pipeline:** parse → type-check (diagnostics only, no runtime effect) → transform (erase types, downlevel decorators/enums/parameter properties) → emit JS (+ optional `.d.ts`); Angular's `ngtsc` adds a template-aware transformation pass on top of vanilla `tsc` , producing AOT-compiled render functions and DI factories at build time — no runtime template compiler ships in production Ivy apps.

Created By - Durgesh Singh